

De : [Ton prénom NOM] <[ton.email]>
À : [Prénom NOM responsable] <[email]>
Objet : Intégration Orderlion ↔ SAP B1 Gervifrais — fiche technique complète

Bonjour [Prénom],

Suite à nos échanges sur les valeurs manquantes côté Orderlion pour la création des BL dans SAP B1, vous trouverez ci-joint la fiche technique complète de l'appel à `POST /Orders` tel qu'attendu par la base Gervifrais.

Le document détaille, champ par champ, la source exacte de chaque donnée — table SAP, requête OData, traitement éventuel — pour que votre équipe puisse implémenter sans devoir deviner. Trois points méritent votre attention particulière :

- **Règle du numéro de lot** (`U_NoLot`) : obligatoire et calculée à partir des bons de réception SAP. Sans elle, le BL est rejeté côté contrôle Gervifrais.
- **Split par entrepôt** : un même article livré depuis deux entrepôts différents doit générer deux lignes distinctes — même `ItemCode`, lots différents.
- **Frais para-fiscaux (INTERFEL + DDG)** : calculés par ligne, avec une étape de réconciliation post-crédation quand le prix est laissé au tarif SAP.

Les identifiants ci-dessous concernent la **base de test** (`GERVIFRAIS_TEST`) — la base de production sera transmise séparément une fois l'intégration validée. Le mot de passe SAP est volontairement omis du document : je vous le communique par canal sécurisé.

Je reste disponible pour clarifier n'importe quel point ou organiser un échange technique avec votre équipe d'intégration. Idéalement, je suggère un premier appel test sur la base `GERVIFRAIS_TEST` avant de passer en prod.

Bien cordialement,
[Ton prénom NOM]
[Téléphone] · [email]

Fiche technique — Création BL SAP B1 pour Orderlion

Cible : **SAP Business One — Service Layer v1** chez Gervifrais. Chaque bloc de code indique son langage en italique au-dessus. Toutes les sources de données sont détaillées au §3.

Environnements : les identifiants ci-dessous sont ceux de la base de **TEST**. La base de production sera fournie séparément par Gervifrais — seuls `SAP_B1_BASE_URL` et `SAP_B1_COMPANY_DB` changent, le reste de la mécanique est identique.

1. Endpoint & authentification

Service Layer SAP B1 en HTTPS sur port dédié. Login → cookie `B1SESSION` (TTL 30 min idle), à renvoyer dans l'en-tête `Cookie` de chaque appel suivant. TLS auto-signé côté SAP : Orderlion doit accepter le certificat (ou désactiver la vérif TLS sur cette base de test uniquement).

Variable	Valeur TEST	Description
<code>SAP_B1_BASE_URL</code>	<code>https://185.57.12.2:40021/b1s/v1</code>	Racine Service Layer (HTTPS, port 40021 sur cette instance)
<code>SAP_B1_COMPANY_DB</code>	<code>GERVIFRAIS_TEST</code>	Nom de la base SAP (multi-tenant)
<code>SAP_B1_USERNAME</code>	<code>GERJMG</code>	Utilisateur SAP dédié à l'interface
<code>SAP_B1_PASSWORD</code>	<i>fourni hors-ligne</i>	À transmettre via canal sécurisé
<code>SAP_B1_TLS_INSECURE</code>	<code>1</code>	Cert auto-signé sur la base TEST → accepter

Langage : HTTP — requêtes

```
# 1) Login (renvoie un cookie B1SESSION valable 30 min)
POST https://185.57.12.2:40021/b1s/v1/Login

# 2) Création de la commande (cookie B1SESSION dans l'en-tête Cookie)
POST https://185.57.12.2:40021/b1s/v1/Orders
```

Langage : JSON — corps du POST /Login

```
{
  "CompanyDB": "GERVIFRAIS_TEST",
  "UserName": "GERJMG",
  "Password": "<fourni hors-ligne>"
}
```

2. Codes fixes Gervifrais

Conventions internes — à coder en dur côté Orderlion, elles ne changent pas entre TEST et PROD.

Entrepôt (WarehouseCode)	Libellé (U_NomMag)	Usage métier
000	A/C - A/D	Réception — à consommer / à dispatcher
01	Stock	Stock physique principal
R1	J+1	Livraison demain

Frais para-fiscal	GroupCode	ExpenseCode	Calcul du LineTotal
INTERFEL (slot TPF2)	1	2	LineTotal_SAP × 0,21 %
Droit De Garde (slot TPF3)	2	3	nb_colis × 0,02 €

Avec `nb_colis = Quantity / SalesQtyPerPackUnit` (= `Quantity` si l'article n'a pas d'emballage défini, c-à-d `SalesQtyPerPackUnit ≤ 1`). Les taux `0,21` et `0,02` sont les valeurs par défaut — la vraie source est `AdditionalExpenses.U_Taux` (cf. §3).

Lot de fallback ultime si aucun PDN trouvé : `EM0000` (variable d'env `GERVIFRAIS_LOT_DEFAULT`).

3. D'où vient chaque champ — map exhaustive

Pour chaque champ du payload `POST /Orders`, voici la source précise : appel OData, table, champ exact.

3.1. Champs doc-level

Champ payload	Source	Comment l'obtenir
CardCode	Orderlion (input) → SAP <code>BusinessPartners.CardCode</code>	Code client SAP fourni par Orderlion. À pré-valider via <code>GET BusinessPartners('XXX')?\$select=CardCode,Frozen,Valid,CreditLimit,CurrentAccountBalance</code> . Refuser si <code>Frozen='tYES'</code> ou <code>Valid='tNO'</code> .
DocDate	Calculé localement	Date du jour <code>YYYY-MM-DD</code> .
DocDueDate	Orderlion (input)	Date de livraison demandée par le client, <code>YYYY-MM-DD</code> .
TaxDate	Calculé localement	= <code>DocDate</code> .
NumAtCard	Orderlion (input)	Référence Orderlion (n° commande client) — visible sur le BL imprimé.
Comments	Orderlion / fixe	Texte libre. Convention : "Via Orderlion – <nom>".
DocumentLines[]	Voir §3.2	Une ligne par couple (<code>ItemCode</code> , <code>WarehouseCode</code>).

3.2. Champs ligne (DocumentLines)

Champ ligne	Source SAP	Comment l'obtenir
ItemCode	Orderlion (input) → SAP <code>Items.ItemCode</code>	Pré-valider via <code>GET Items('XXX')?\$select=ItemCode</code> . Évite l'erreur SAP cryptique <code>-2028</code> .

Champ ligne	Source SAP	Comment l'obtenir
Quantity	Orderlion (input)	En unité de stock SAP (champ InventoryUOM de l'item, généralement pie). Si Orderlion travaille en colis, convertir : $qte_pie = qte_colis \times SalesQtyPerPackUnit$.
WarehouseCode	Orderlion (input) + check stock	Un de 000 , 01 , R1 . Vérifier la dispo via GET Items('XXX') ? \$select=ItemWarehouseInfoCollection → champ InStock - Committed par entrepôt.
UnitPrice (et/ou Price)	Orderlion (input)	Prix HT par unité de stock SAP. Si omis, SAP applique son tarif client et il faut réconcilier les TPF au §7.
U_NomMag	Calculé localement	Libellé humain de WarehouseCode , cf. table §2 : {000: "A/C - A/D", 01: "Stock", R1: "J+1"} .
U_NoLot	SAP PurchaseDeliveryNotes	Cf. §4. "EM" + DocNum du dernier PDN contenant l'article dans cet entrepôt. Champ obligatoire côté Gervifrais.
U_GER_Pays	SAP Items.U_Pays	GET Items('XXX')?\$select=U_Pays . À pré-charger / cacher.
U_GER_Marque	SAP Items.U_GER_Marque	GET Items('XXX') ? \$select=U_GER_Marque .
U_GER_Condi	SAP Items.U_GER_Det_Condt	⚠ Renommage : champ U_GER_Det_Condt côté Items, à poser comme U_GER_Condi sur la ligne de la commande. Ex. "12x125g" .
DocumentLineAdditionalExpenses[]	Calculé + SAP AdditionalExpenses	Voir §3.3.

3.3. Frais para-fiscaux (TPF par ligne)

Sous-champ	Source	Comment l'obtenir
GroupCode	Fixe (cf. §2)	1 pour INTERFEL, 2 pour DDG.
ExpenseCode	SAP AdditionalExpenses.ExpensCode	⚠ Typo SAP : le master expose ExpensCode (sans 'e' entre s et C). À la ligne le champ est ExpenseCode (avec). Valeurs Gervifrais : 2 =INTERFEL, 3 =DDG. Lecture : GET AdditionalExpenses?\$top=50 .
LineTotal (INTERFEL)	Calculé : HT × taux	Taux dans AdditionalExpenses(ExpensCode=2).U_Taux — valeur courante 0,21 (= 0,21 %). Formule : $round(LineTotal_SAP \times 0.0021, 2)$. Le LineTotal_SAP n'est connu qu'après création → réconciliation au §7.
LineTotal (DDG)	Calculé : nb_colis × taux	Taux dans AdditionalExpenses(ExpensCode=3).U_Taux — valeur courante 0,02 €. Formule : $round((Quantity / SalesQtyPerPackUnit) \times 0.02, 2)$. SalesQtyPerPackUnit vient de GET Items('XXX') ? \$select=SalesQtyPerPackUnit .

3.4. Données à pré-charger / cacher

Donnée	Appel SAP	TTL conseillé
Master AdditionalExpenses (taux TPF)	GET AdditionalExpenses?\$top=50	10 min

Donnée	Appel SAP	TTL conseillé
Index des derniers PDN par (ItemCode, WarehouseCode)	GET PurchaseDeliveryNotes? \$top=50&\$skip=N&\$orderby=DocNum desc&\$select=DocNum,DocumentLines (scan 500 docs)	10 min
Méta-article (U_Pays , U_GER_Marque , U_GER_Det_Condt , SalesQtyPerPackUnit , SalesUnitWeight , InventoryUOM)	GET Items?\$filter=Valid eq 'tYES' and Frozen eq 'tNO'&\$select=... (paginé, ~1300 articles)	1 h ou sync 1×/jour
État stock par entrepôt (ItemWarehouseInfoCollection)	GET Items('XXX')? \$select=ItemWarehouseInfoCollection (par item, à la volée)	30 s (sync delta)

4. Règle U_NoLot (numéro de lot)

Obligatoire — un BL sans U_NoLot conforme passe côté SAP mais est rejeté au contrôle Gervifrais. Règle confirmée par probe sur des BL réels.

Langage : Pseudo-code — règle métier

```
U_NoLot = "EM" + DocNum du DERNIER PurchaseDeliveryNote
           contenant l'ItemCode DANS le WarehouseCode de la ligne

# Cascade de fallback si rien trouvé :
# 1. (ItemCode + WarehouseCode) → cas nominal
# 2. (ItemCode, dernière réception toutes confondues)
# 3. "EM0000" (constante GERVIFRAIS_LOT_DEFAULT)

# Exemples observés sur la base TEST :
# FE1SL reçu en 000 via PDN n°22739 → U_NoLot = "EM22739"
# FE1SL reçu en 01 via PDN n°22755 → U_NoLot = "EM22755"
```

Conséquence métier : un même ItemCode livré depuis 2 entrepôts différents = 2 lots différents = **2 lignes distinctes** dans DocumentLines . Le split se fait dans l'input Orderlion, pas dans le builder de payload.

5. Payload complet de POST /Orders

Exemple : client APLAI , 30 pièces de FE1SL réparties entre 2 entrepôts (20 en 01 , 10 en 000) → 2 lignes.

Langage : JSON — corps du POST /Orders

```

{
  "CardCode": "APLAI",
  "DocDate": "2026-06-03",
  "DocDueDate": "2026-06-04",
  "TaxDate": "2026-06-03",
  "NumAtCard": "ORD-12345",
  "Comments": "Via Orderlion",

  "DocumentLines": [
    {
      // — Ligne 1 : 20 pie depuis le stock physique (01) —
      "ItemCode": "FE1SL",
      "Quantity": 20,
      "WarehouseCode": "01",
      "U_NomMag": "Stock",
      "U_NoLot": "EM22755", // dernier PDN de FE1SL en 01
      "U_GER_Pays": "FRANCE", // = Items.U_Pays
      "U_GER_Marque": "GERVI", // = Items.U_GER_Marque
      "U_GER_Condi": "12x125g", // = Items.U_GER_Det_Condt

      "DocumentLineAdditionalExpenses": [
        { "GroupCode": 1, "ExpenseCode": 2, "LineTotal": 8.40 }, // INTERFEL
        { "GroupCode": 2, "ExpenseCode": 3, "LineTotal": 0.80 } // DDG
      ]
    },
    {
      // — Ligne 2 : 10 pie depuis A/C-A/D (000) —
      "ItemCode": "FE1SL",
      "Quantity": 10,
      "WarehouseCode": "000",
      "U_NomMag": "A/C - A/D",
      "U_NoLot": "EM22739", // dernier PDN de FE1SL en 000
      "U_GER_Pays": "FRANCE",
      "U_GER_Marque": "GERVI",
      "U_GER_Condi": "12x125g",

      "DocumentLineAdditionalExpenses": [
        { "GroupCode": 1, "ExpenseCode": 2, "LineTotal": 4.20 },
        { "GroupCode": 2, "ExpenseCode": 3, "LineTotal": 0.20 }
      ]
    }
  ]
}

```

Réponse SAP utile : `DocEntry` (id technique), `DocNum` (n° de doc visible), `DocTotal` (TTC), `VatSum` (TVA), `DocumentLines[].LineTotal` (HT calculé par SAP, utile pour la réconciliation \$7).

6. Code minimal — split par lot

Le split (un même article sur 2 entrepôts → 2 lignes) est **dans l'input Orderlion**. Le builder ne fait que mapper chaque ligne d'input vers une ligne SAP enrichie.

Langage : TypeScript — pré-calcul du lot (cache 10 min)

```

// Construit deux index à partir des 500 dernières réceptions :
// byItemWhs : "ItemCode|Warehouse" → dernier DocNum vu (cas nominal)
// byItem    : "ItemCode"          → dernier DocNum vu (fallback)
async function buildLotMaps() {
  const byItemWhs = new Map<string, number>();
  const byItem    = new Map<string, number>();

  for (let skip = 0; skip < 500; skip += 50) {
    const r = await sapGet(
      "PurchaseDeliveryNotes"
      + "?$top=50&$skip=" + skip
      + "&$orderby=DocNum desc"
      + "&$select=DocNum,DocumentLines"
    );
    if (!r.value?.length) break;

    for (const doc of r.value) {
      for (const line of doc.DocumentLines ?? []) {
        if (!line.ItemCode) continue;

        // On garde le DocNum max par sécurité (au cas où l'ordre flotte)
        const prevItem = byItem.get(line.ItemCode) ?? 0;
        if (doc.DocNum > prevItem) byItem.set(line.ItemCode, doc.DocNum);

        if (line.WarehouseCode) {
          const key = line.ItemCode + "|" + line.WarehouseCode;
          const prev = byItemWhs.get(key) ?? 0;
          if (doc.DocNum > prev) byItemWhs.set(key, doc.DocNum);
        }
      }
    }
  }
  return { byItemWhs, byItem };
}

```

Langage : TypeScript — résolution du lot pour une ligne donnée

```

// Cascade : (item+entrepôt) → (item seul) → fallback constant.
function resolveLot(
  maps: { byItemWhs: Map<string, number>; byItem: Map<string, number> },
  itemCode: string,
  warehouseCode: string,
): string {
  // 1. Cas nominal : réception vue pour cet item dans cet entrepôt
  const exact = maps.byItemWhs.get(itemCode + "|" + warehouseCode);
  if (exact) return "EM" + exact;

  // 2. Fallback : item vu ailleurs → dernière réception toutes confondues
  const any = maps.byItem.get(itemCode);
  if (any) return "EM" + any;

  // 3. Fallback ultime (cf. GERVIFRAIS_LOT_DEFAULT)
  return "EM0000";
}

```

Langage : TypeScript — construction du payload final (avec split)

```
// Libellés humains des entrepôts (cf. §2)
const WHS_NAMES = {
  "000": "A/C - A/D",
  "01": "Stock",
  "R1": "J+1",
} as const;

// Une ligne d'input Orderlion = un couple (article, entrepôt)
type CartLine = {
  itemCode: string;
  quantity: number; // en unité de stock SAP (pie)
  warehouseCode: "000" | "01" | "R1";
};

async function buildOrderPayload(
  cart: CartLine[],
  cardCode: string, // code client SAP (BusinessPartners.CardCode)
  dueDate: string, // "YYYY-MM-DD"
  ref: string, // référence Orderlion → NumAtCard
) {
  const lotMaps = await buildLotMaps();

  // Cache local des méta-articles : U_Pays, U_GER_Marque, U_GER_Det_Condt,
  // SalesQtyPerPackUnit. Lecture batch via Items?$filter=...
  const items = await loadItemMeta(cart.map(l => l.itemCode));

  const documentLines = cart.map((l) => {
    const meta = items.get(l.itemCode);
    return {
      ItemCode: l.itemCode,
      Quantity: l.quantity,
      WarehouseCode: l.warehouseCode,
      U_NomMag: WHS_NAMES[l.warehouseCode],
      U_NoLot: resolveLot(lotMaps, l.itemCode, l.warehouseCode),
      // Renommage Items → ligne : U_GER_Det_Condt devient U_GER_Condi
      ...(meta?.U_Pays && { U_GER_Pays: meta.U_Pays }),
      ...(meta?.U_GER_Marque && { U_GER_Marque: meta.U_GER_Marque }),
      ...(meta?.U_GER_Det_Condt && { U_GER_Condi: meta.U_GER_Det_Condt }),
      // TPF : voir §7 pour le calcul et la réconciliation post-crédation
    };
  });

  const today = new Date().toISOString().slice(0, 10);
  return {
    CardCode: cardCode,
    DocDate: today,
    DocDueDate: dueDate,
    TaxDate: today,
    NumAtCard: ref,
    Comments: "Via Orderlion",
    DocumentLines: documentLines, // N lignes, une par (item, entrepôt)
  };
}

```

7. Réconciliation TPF après création

Si le prix est laissé au tarif SAP, le HT (`LineTotal`) n'est connu qu'*après* la création de la commande. Il faut donc **relire** la commande puis **PATCH**er les TPF par ligne avec les vraies valeurs. Sans ça, INTERFEL est à 0 ou faux.

Langage : HTTP — séquence des appels

```
# 1) Relire la commande qu'on vient de créer
GET /Orders({DocEntry})

# 2) PATCH ciblé des DocumentLineAdditionalExpenses par ligne
PATCH /Orders({DocEntry})

```

Langage : TypeScript — recalcul et PATCH

```

// Taux lus depuis AdditionalExpenses (mis en cache 10 min)
// AdditionalExpenses(ExpensCode=2).U_Taux → INTERFEL
// AdditionalExpenses(ExpensCode=3).U_Taux → DDG
const ITFEL_RATE = 0.0021; // 0,21 % du LineTotal SAP
const DDG_RATE = 0.02; // 0,02 € par colis

async function reconcileTpf(docEntry: number) {
  // 1. Relire : SAP a calculé le vrai LineTotal de chaque ligne
  const order = await sapGet("Orders(" + docEntry + ")");

  const patchLines = [];
  for (const line of order.DocumentLines) {
    const meta = await getItemMeta(line.ItemCode);
    const packDiv = meta?.SalesQtyPerPackUnit > 1
      ? meta.SalesQtyPerPackUnit
      : 1;
    const nbColis = (line.Quantity ?? 0) / packDiv;

    // Valeurs correctes attendues
    const itfel = round2((line.LineTotal ?? 0) * ITFEL_RATE);
    const ddg = round2(nbColis * DDG_RATE);

    // Valeurs actuellement posées sur la ligne
    const exps = line.DocumentLineAdditionalExpenses ?? [];
    const curItfel = exps.find(e => e.ExpenseCode === 2)?.LineTotal ?? 0;
    const curDdg = exps.find(e => e.ExpenseCode === 3)?.LineTotal ?? 0;

    // Si écart > 1 centime → PATCH cette ligne
    if (Math.abs(itfel - curItfel) > 0.005
      || Math.abs(ddg - curDdg) > 0.005) {
      const merged = [];
      if (itfel > 0) merged.push(
        { GroupCode: 1, ExpenseCode: 2, LineTotal: itfel });
      if (ddg > 0) merged.push(
        { GroupCode: 2, ExpenseCode: 3, LineTotal: ddg });
      patchLines.push({
        LineNum: line.LineNum,
        DocumentLineAdditionalExpenses: merged,
      });
    }
  }

  if (patchLines.length > 0) {
    await sapPatch("Orders(" + docEntry + ")",
      { DocumentLines: patchLines });
  }
}

const round2 = (n: number) => Math.round(n * 100) / 100;

```

8. Pièges connus

- **Typo SAP ExpensCode vs ExpenseCode** : le master AdditionalExpenses expose ExpensCode (sans 'e'), alors qu'à la ligne le champ est ExpenseCode (avec). Les deux orthographes sont nécessaires.
- **Renommage U_GER_Det_Condt → U_GER_Condi** : le champ master article et le champ ligne n'ont pas le même nom. À mapper explicitement.
- **U_NoLot manquant ou "EM0000"** : passe côté SAP mais sera bloqué au contrôle Gervifrais. Logger un warning quand le fallback est utilisé.
- **Session SAP** : idle > 30 min → 401 Unauthorized. Prévoir re-login automatique avec un verrou pour éviter les logins parallèles concurrents (sinon n logins simultanés = saturation).
- **TLS auto-signé (base TEST)** : certif non chaîné. Côté Orderlion : accepter le cert (pinning) ou désactiver la vérif TLS sur cette base uniquement.
- **WarehouseCode hors { 000, 01, R1 }** : SAP l'accepte mais l'article sort du flux télévente standard. Refuser en validation Orderlion.
- **Pré-validation conseillée avant POST** : GET Items('XXX') pour chaque ItemCode unique + GET BusinessPartners('YYY'). Évite l'erreur SAP -2028 (article inexistant) et permet un message d'erreur clair côté Orderlion.

- **Quantités :** SAP attend la quantité en unité d'inventaire (InventoryUOM , généralement pie). Si Orderlion travaille en colis, convertir avant POST.